ParkiStep Twin - implemented architecture

Live prediction uses only past sensor data (blue). Annotations (red, dashed) train the model offline and score replays after they end (purple, dashed); they never reach live inference.

Legend

solid blue	live sensor data (past only)
solid green	historical clinical context
solid orange	model artifacts / model outputs
dashed red	training-only annotations
dashed purple	retrospective evaluation / ground truth
dotted grey	optional persistence

Source data
(FoG-STAR, not redistributed)

annotation columns
(FoG flag, severity, activity)
from video - labels only

sensor_data.csv
wearable accelerometer / gyroscope
channels, 4 body positions

clinical_data.csv
one record per participant

Phase 1
data foundation

ingestion +
schema validation
(parkistep.ingestion, validation)

quality audit + join audit
(parkistep.quality, integration)
PASS_WITH_WARNINGS

processed tables
data/processed/fogstar
(participant-level fusion key)

TRAINING-ONLY labels

ground truth

sensor + clinical
(training windows)

ground truth,
after replay only

Phase 2
forecasting
(offline)

windows + events
(forecast.windows, events)
2.0 s input, 0.2 s stride;
label = onset within horizon H

features
(forecast.features)
sensor statistics
(+ clinical fields for fusion)

models + experiment
(forecast.models, experiment)
logistic, random forest, dummies;
participant-grouped 5-fold CV

MODEL ARTIFACTS
*.joblib + *.manifest.json
SHA-256, feature schema, horizon,
threshold + provenance

out-of-fold
predictions

historical
clinical record

LIVE: past samples only

out-of-sample evaluation
(forecast.evaluate)
event sensitivity, false alerts/h,
lead time, bootstrap CIs

load + verify

Phase 3
digital-twin
runtime

retrospective comparison
(twin/replay.py)
runs only AFTER the replay ends

warnings + scores

warning manager
(twin/alerts.py)
threshold, persistence, merge;
stale / unavailable states

score
(uncalibrated)

model gateway
(twin/model.py)
verifies manifest + SHA-256,
schema, horizon -> ForecastPredictor

trailing window

twin state + rolling buffer
(twin/state.py, buffer.py)
clinical profile = historical context

observations

streams
(twin/streams.py)
replay one recording sample by
sample; annotations withheld

warning state

saved Phase 2 report

Phase 4
dashboard

Streamlit app - 7 pages
Overview, Patient Profile, Movement Monitor,
Early Warning, Warning Center,
Replay Analysis, Model Performance

DashboardService + ReplaySession
(dashboard/service.py, replay_session.py)
start / pause / resume / stop; failed state

SQLite state store (optional)
(twin/storage.py)
twin state, predictions, events